MSMK

FRONT PAGE

Nombre Alumno / DNI	Andrés Ramirez Carroll 54977572M
Título del Programa	PAE Cibersecurity and Digital Intelligence
N.º Unidad y Título	UNIT 15 - Artificial Intelligence & Intelligent Systems -
Año académico	2025-2026
Profesor de la unidad	José Francisco Ibañez
Título del Assignment	AB Final
Día de emisión	23/10/2025
Día de entrega	29-01-2026
Declaración del estudiante	Certifico que la presentación del assignment es completamente mi propio trabajo y entiendo completamente las consecuencias del plagio. Entiendo que hacer una declaración falsa es una forma de mala práctica. **Fecha:** 29-01-2026 **Firma del alumno:**

Plagio

El plagio es una forma particular de hacer trampa. El plagio debe evitarse a toda costa y los alumnos que infrinjan las reglas, aunque sea inocentemente, pueden ser sancionados. Es su responsabilidad asegurarse de comprender las prácticas de referencia correctas. Como alumno de nivel universitario, se espera que utilice las referencias adecuadas en todo momento y mantenga notas cuidadosamente detalladas de todas sus fuentes de materiales para el material que ha utilizado en su trabajo, incluido cualquier material descargado de Internet. Consulte al profesor de la unidad correspondiente o al tutor del curso si necesita más consejos.

INDICE

1. Marco teórico.
2. Ingesta de datos.
3. Exploración y limpieza.
4. Linea temporal.
5. Machine learning.
6. Predicciones.
7. Preguntas.
8. Conclusiones.
9. Bibliografia.

MARCO TEÓRICO

Pandas: Es una Biblioteca/Librería de código abierto, esta diseñada para la manipulación y análisis de datos, se pueden cargar, alinear, manipular o fusionar datos(Liora,s.f.b).

Uso: En este proyecto he usado pandas para poder transformar, manipular, explorar y limpiar los datos entregados.

Matplotlib: Es una Biblioteca/Librería de open source, originalmente se oriento su uso para visualizar las señales eléctricas del cerebro de personas epilépticas, a lo largo del tiempo ha ido evolucionando y ahora se utiliza para crear gráficas y diagramas de gran calidad (Liora, s.f.d).

Uso: En este proyecto la he utilizado para crear las lineas temporales y las gráficas utilizadas por prophet para las predicciones.

Numpy: Es una Biblioteca/Librería de código abierto la cual es una abreviatura de (Numerical Python), se utiliza para la programación científica, la programación en DataScience, la ingeniería y las matemáticas o ciencias (Liora, s.f.a).

Uso: En este proyecto no he utilizado numpy directamente, pero lo he usado como motor de computación para ayudar a la librería pandas a realizar operaciones aritméticas mas eficientes.

Seaborn: Es una Biblioteca/Librería utilizada para la creación de gráficos estadísticos, esta basada en matplotlib y se integra con la estructura de pandas (Liora, s.f.c).

Uso: En este proyecto la he utilizado para crear una gráfica de barras en la comparación de los embalses.

Facebook Prophet: Es una Biblioteca/Librería de código abierto desarrollada por Facebook, esta librería es usada por su automatización de los procesos rutinarios de los métodos de previsión tradicionales, de esta manera ayuda a los usuarios a producir rápidamente previsiones interpretables y de gran calidad (DataCamp, s.f.a).

Uso: En este proyecto la he utilizado para hacer las predicciones sobre los embalses en un rango temporal de 12 meses.

INGESTA DE DATOS

```python
# Definiremos una variable a la que llamaremos datos_aeme
def datos_aeme():
    # Creamos una lista para copiar todas las rutas de los archivos.
    rutas = [
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioCofio_LaAcena.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioGuadalix_Pedrezuela.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioGuadarrama-Aulencia_LaJorosa.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioGuadarrama-Aulencia_Navalmedio.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioGuadarrama-Aulencia_Valmayor.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioJarama_ElVado.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLosMorales_LosMorales.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLozoya_ElAtazar.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLozoya_ElVillar.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLozoya_LaPinilla.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLozoya_PuentesViejas.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioLozoya_Riosequillo.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioManzanares_Navacerrada.csv',
    '/content/drive/MyDrive/AB01_IA/csv/csv/AguaEmbalsada_RioManzanares_Santillana.csv'
    ]
# Creamos una lista vacia para que pueda guardar temporalemte los csv que esten validados.
    l_df= []
# Creamos un bucle que recorra la lista llamda rutas.
    for x in rutas:
      # Le pedimos que añada a l_df los csv que esten validados por (sep = ';' esto es para que busque en el csv y si tiente algun separador de punto y coma no se lo salte) y (encoding = 'Latin-1' esto es para que que el codigo pueda leer las tildes
      # He decido añadir un dataframe temporal para crear una columna 'x' de esta manera se guarda el punto de origen de los embalses y no se pierden.
        try:
          df_temporal = pd.read_csv(x,sep = ';', encoding = 'Latin-1')
          df_temporal['Nombre_embalse'] = x
          l_df.append(df_temporal)
        # Pedimos un print para saber si hay algun problema a la hora de leer los csv.
          print('Todo bien: ' + x)
      # Hacemos una excepcion y la apodamos como 'e', en el print de acontinuacion nos dira que ruta del csv y que esta mal.
        except Exception as e:
          print(f'Esto esta mal {x}: {e}')
# Creamos una variable llamda df_final en la que usaremos la funcion 'pd.concat' de pandas para poder unir todos los csv en uno, 'ignore_index=True' esto es para que cuando los una no acabe un csv en el indice 12 y el siguiente empiece por el 1 si no
    df_final=pd.concat(l_df, ignore_index=True)
    return df_final
# Creamos una ultima variable llamada df_sequia y la igualamos a la anterior variable llamada datos_aeme, y pedimos un print para poder visualizar todo.
df_sequia = datos_aeme()
print(df_sequia)
```

Para la correcta y limpia ingesta de datos para este proyecto, he decidido automatizar el proceso mediante un bucle, una excepción, una devolución y un conjunto de funciones de pandas.

Bucle for: Lo he utilizado para poder recorrer la lista llamada rutas.

Try/Except: He utilizado esta función para poder tener un estándar de seguridad, esta función permite que el código siga funcionando después de encontrar errores, como modo de información del curso del mismo he implementado el print('Todo bien' + x) el cual nos indica si hay error o no.

Lectura de los csv: Para una correcta lectura de los cvs he utilizado la función 'pd.read' de la librería pandas para leer los archivos,he añadido el separador ';' para poder leer correctamente los csv y le he añadido la función encoding 'Latin-1'.

Latin-1: O también conocido como ISO8859-1 es un estándar de codificación de caracteres ampliamente utilizados que permite representar textos en idiomas europeos occidentales en este caso lo he utilizado para el español.

PD.read: Es el método principal para poder importar los archivos csv a un dataframe.

Unión de los datos: Para poder tener un solo dataframe de los datos en el que siguen un orden he utilizado la función 'pd.concat', en conjunto con el delimitador 'ignore_index =

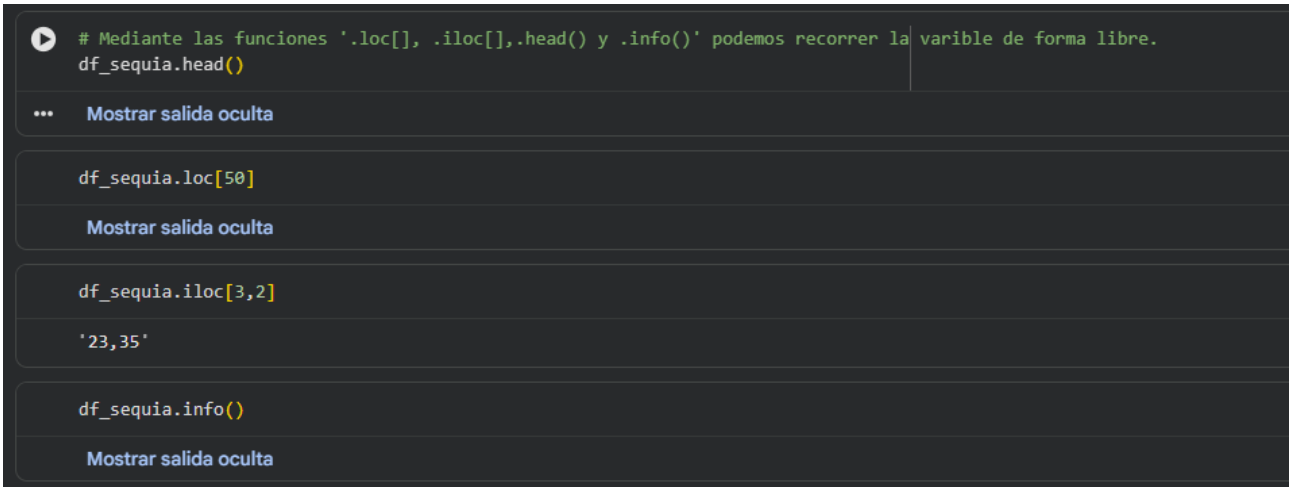
True’ esto une los csv y cuando el primero termina el siguiente sigue el ultimo indice, es decir si el primero acaba en el 40 la función permite que el siguiente fila empiece en el 41.

PD.concat: Esta función es la responsable de concatenar los archivos. Ignore_index = True: La función principal controla el indice mientras que si añadimos ‘Flase’ el archivo no unificara los indices de todos los datos, mientras tanto si usamos ‘True’ crea automáticamente un indice para cada fila que concatena.

Return: Se ha utilizado para que el bucle nos devuelva los datos guardados en un dataframe llamado df_final.

EXPLORACIÓN Y LIMPIEZA

Como ya tenemos los datos almacenados en un solo dataframe, nos disponemos a explorarlo esto lo he realizado mediante las funciones: .head(), .loc[], .iloc[].



Head(): La función devuelve los primeros elementos de la estructura, ya sea para series o para dataframe, por defecto son los 5 primeros.

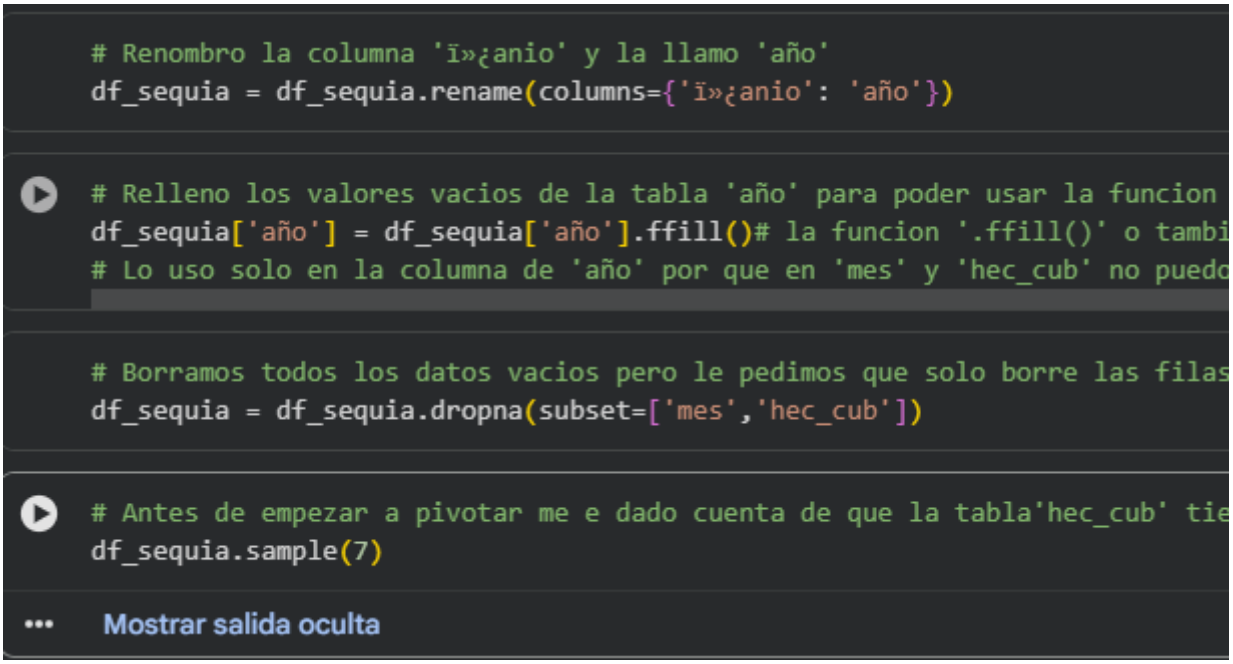
Uso: En esto caso se ha utilizado para la exploración de un dataframe.

Loc[]: Es un metodo para seleccionar y filtrar datos de un dataframe basado en etiquetas.

Uso: En este caso se ha utilizado para una exploración precisa buscando datos por el indice.

Iloc[]: Metodo de busca basado en la poscicion.

Uso: En este caso se ha utilizado para obtener una exploración muy detallada buscando por filas y columnas del dataframe.



Al explorar el dataframe me percate de 3 principales problemas con el dataframe.

1- La columna año tenia un nombre poco claro.
2- La columna año solo tenia datos en la primera fila de cada bloque de años.
3- El dataframe estaba lleno de valores que no aportaban información util.

Soluciones:

Renombrado de la columna: Renombramos la columna a ‘año’ esto se hace posible gracias la función ‘.rename()’

Relleno de los datos: Para poder rellenar esas filas vacias de la columna año, he usado la función ‘.ffill()’, esta función lee toda la columna y cuando encuentra una fila vacia le pone el valor que tenia la anterior.

Eliminación de los datos nulos: Para poder limpiarlo de ‘Nan’ he usado la función ‘.dropna()’ la cual elimina todos los valores Nan de las filas, ponemos una excepción a mes y hec_cub por que no queremos borrar las columnas que no tienen nombre.

Rename: Es el método mas seguro y explicito para el re-nombramiento de columnas.

Ffill: Este método reemplaza el valor nulo con el valor de la anterior fila o columna si el parámetro del axis esta delimitado a ‘columns’.

Dropna: Filtra los valores de una estructura de datos para dejar solo los valores no nulos. Añadiendo ‘subset’ especificamos de donde queremos que borre los Nan.

```
# Como quiero pivotar la tabla 'hec_cub' y abajo, me he dado cuenta de que el separador de los num
df_sequia['hec_cub'] = pd.to_numeric(df_sequia['hec_cub'].str.replace(',','.'), errors ='coerce')
df_sequia = df_sequia.dropna(subset=['hec_cub'])
```

```
# Para poder pivotar sin preocupaciones, me he creado un 'df' llamado 'df_final' en el que he ejut
# Como ha salido bien 'df_final' va a ser el dataframe que voy a utilizar ya que las tablas que ne
df_final=pd.pivot_table(df_sequia,
                index=('año','mes'),
                columns='Nombre_embalse',
                values='hec_cub',
                aggfunc='sum')
df_final = df_final.fillna(0)
df_final['Total']=df_final.sum(axis=1)
df_final.head()
```

Mostrar salida oculta

Pasos siguientes: Generar código con df_final | New interactive sheet

```
# Para limpiar la tabla de 'Embalse' uso el metodo '.split' con el separador '/' y '.replace' para
df_final.columns=df_final.columns.str.split('/').str[-1] # el '[-1]' lo uso para que la funcion '.
df_final.columns=df_final.columns.str.replace('.csv','',regex=False) # el 'regex=False' lo uso par
df_final.columns=df_final.columns.str.replace('AguaEmbalsada_','',regex=False)
df_final.head()
```

Mostrar salida oculta

Pasos siguientes: Generar código con df_final | New interactive sheet

```
# Para resetear el 'indice' voy a usar la funcion '.reset_index()' sin los siguiente parametros 'd
# Ya que pandas lo usa por defecto.
df_final=df_final.reset_index()
```

Antes de reestructurar la tabla, detecte que la columna ‘hec_cub’ se estaba detectando como ‘str’ esto ocurría por que el separador de los decimales es la ‘,’ y no el ‘.’.

Conversión y reemplazo: Usamos la función pd.to_numeric para cambiar la columna a float y usamos la función ‘.replace’ para cambiar la ‘,’ por un ‘.’.

Conversion de errores: Use el parámetro ‘errors = ‘coerce’’ por si se encontraban datos que no fuesen números lo convirtiese directamente a Nan.

Volvemos a usar la función ‘.dropna’ para eliminar los valores Nan.

PD.to_numeirc: Convierte el argumento a numerico.

Errors = coerce: Convierte a Nan todos los errores.

Replace: Cambia el valor marcado por el especificado.

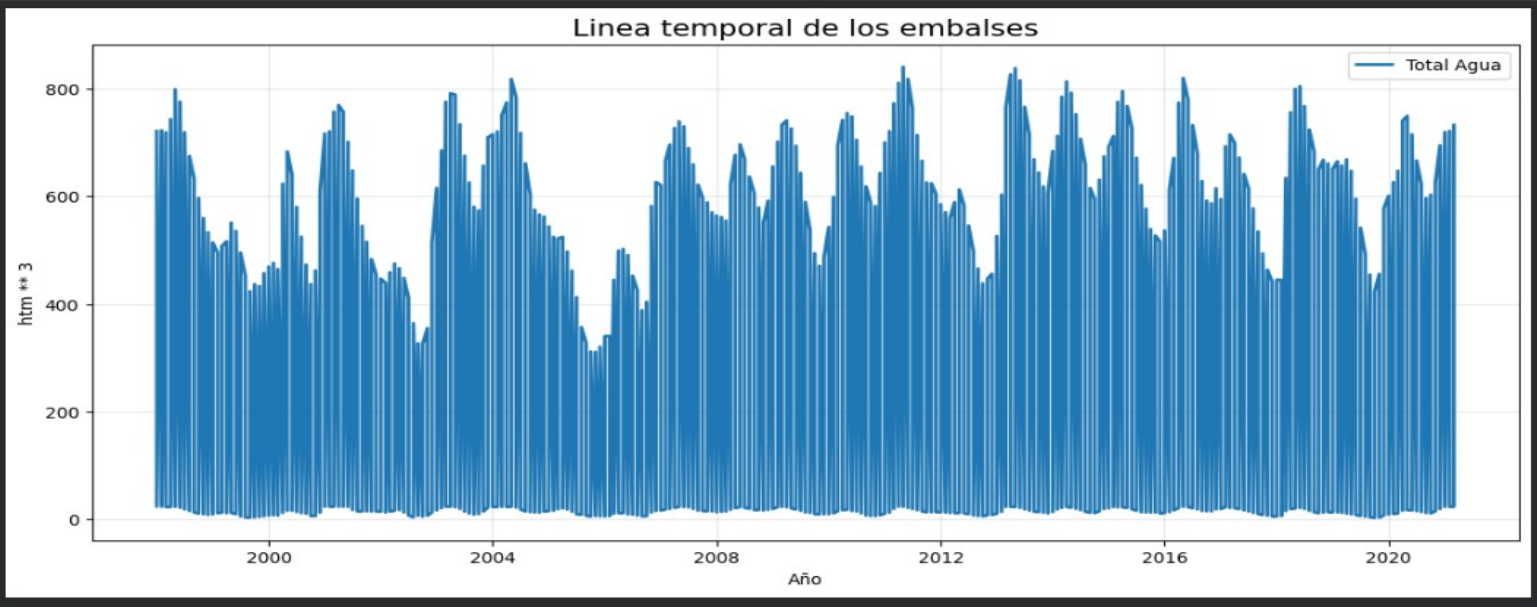
Para poder hacer la comparación de los embalses era necesario reestructurar el dataframe por ello use la función ‘pd.pivot_table()’ que permite reoganizar los datos.

Converti las columnas de año y mes para que fuesen el indice, y puse los nombres de los embalses como columnas.
Como valor a mostrar use la columna de los ‘hec_cub’, como medida de protección he usado ‘fillna(0)’ para que rellene los valores Nan por el valor anteriormente validado.

PD.pivot_table: Es la funcion que permite delimitar los objetos mediante un Multiindex.

LINEA TEMPORAL

```
if not top.empty and not df_grafica.empty: # He repetido la creación de la linea temporal, pero esta vez he incluido la variable 'top' ya que los datos s
  plt.figure(figsize=(14, 6)) # '.empty' nos aseguramos de que no esten vacios, ya que anteriormente lo intente con 'if top > 0 and df_grafica > 0' y dio
  plt.plot(df_grafica['Fecha'], df_grafica['Total'], color='#1f77b4', linewidth=2, label='Total Agua')
  plt.title('Linea temporal de los embalses', fontsize=16)
  plt.xlabel('Año')
  plt.ylabel('htm ** 3')
  plt.grid(True, alpha=0.3)
  plt.legend()
  plt.show()
```



Tras haber limpiado todos los datos y haber realizado una validación de los mismos con la variable llamada ‘top’, he representado una gráfica de la evolución de todos los embalses a lo largo del tiempo.

La gráfica se lee de la siguiente forma:

El eje x el cual representa los años, el último es hasta cuando se han recogido los datos.

El eje y el cual representa los hm^3 , son los máximos y mínimos hasta donde han llegado los niveles de agua en los embalses.

Las barras azules representan a nuestros embalses, según su altura el conglomerado de embalses ha tenido más agua o menos.

Se puede observar que las barras mas altas representas los periodos de lluvias invierno y primavera, mientras que las mas bajas representan los periodos menguantes el verano.

MACHINE LEARNING

Para poder hacer las predicciones, he decido utilizar Facebook prophhet. La cual es una herramienta muy potente la hora de hacer predicciones y gráficas fáciles de interpretar con una calidad muy superior.

```
modelo_total=Prophet(seasonality_mode= 'multiplicative', yearly_seasonality = True) # Repetimos las acciones usadas en la prediccion del embalse anterior
modelo_total.fit(df_Totalprophet)
total_futuro = modelo_total.make_future_dataframe(periods=12, freq='MS')
prediccion_total=modelo_total.predict(total_futuro)
```

```
INFO:prophet:Disabling weekly seasonality. Run prophet with weekly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
```

He decido usar el modo multiplicativo sobre el aditivo por que: El multiplicativo muestra un resultado del crecimiento porcentual, el cual representa unas tendencias del dato mas exponenciales.

En cambio el modo aditivo es el resultado de la suma de los valores numéricos y representa unas tendencias del dato mas lineales.

La configuración de la estacionalidad se ha activado mediante el cambio a 'yearly_seasonality = True' esta función es crucial por que nosotros queremos hacer una predicción de 12 meses los cuales equivalen a un año, al ejecutar el modelo prophet detecto que en nuestros datos no hay registros diarios ni semanales y por ello desactivo automáticamente las predicciones diarias y semanales las cuales solo le darían información errónea y desafía nuestro objetivo.

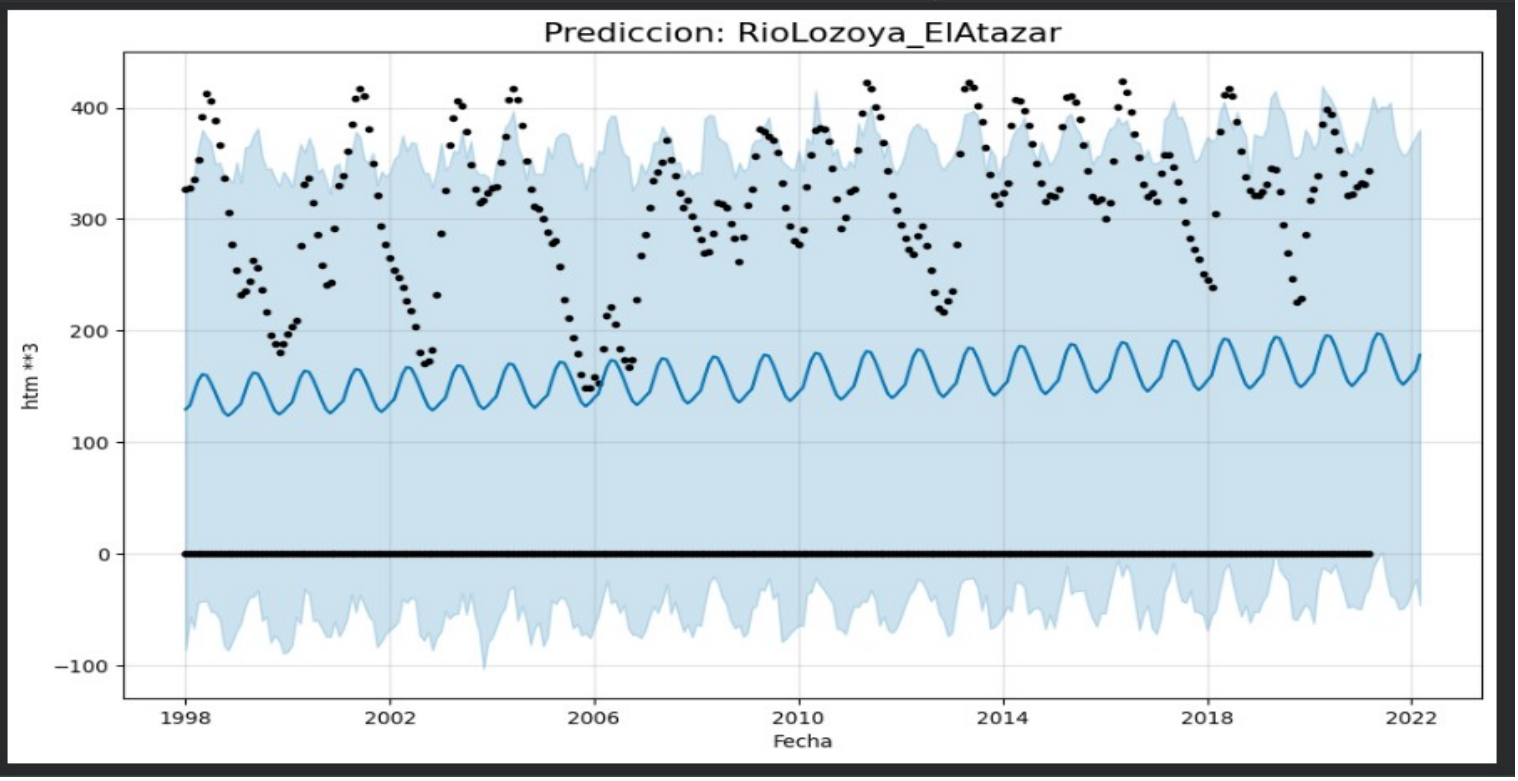
Para que prophet tenga datos con los que predecir, se usa la función '.fit()' la cual es la encargada de ejecutar el modelo de aprendizaje: procesa los datos históricos y calcula los parámetros almacenándolos en memoria para poder realizar predicciones futuras.

Para configurar las tablas sobre las que queremos hacer las predicciones, hay que usar el metodo 'make_future_dataframe()' como queremos hacer una prediccion de 12 meses se lo indicamos en el campo de periodos, en el campo de frecuancia indicamos 'MS' que son las siglas de Month Start para que los datos sean del primero de cada mes.

PREDICCIONES

Una vez que tenemos el modelo ya entrenado, le creado visualizaciones a través de gráficas para poder interpretar las predicciones. Prophet a generado un total de 3 gráficas para poder entender la serie temporal.

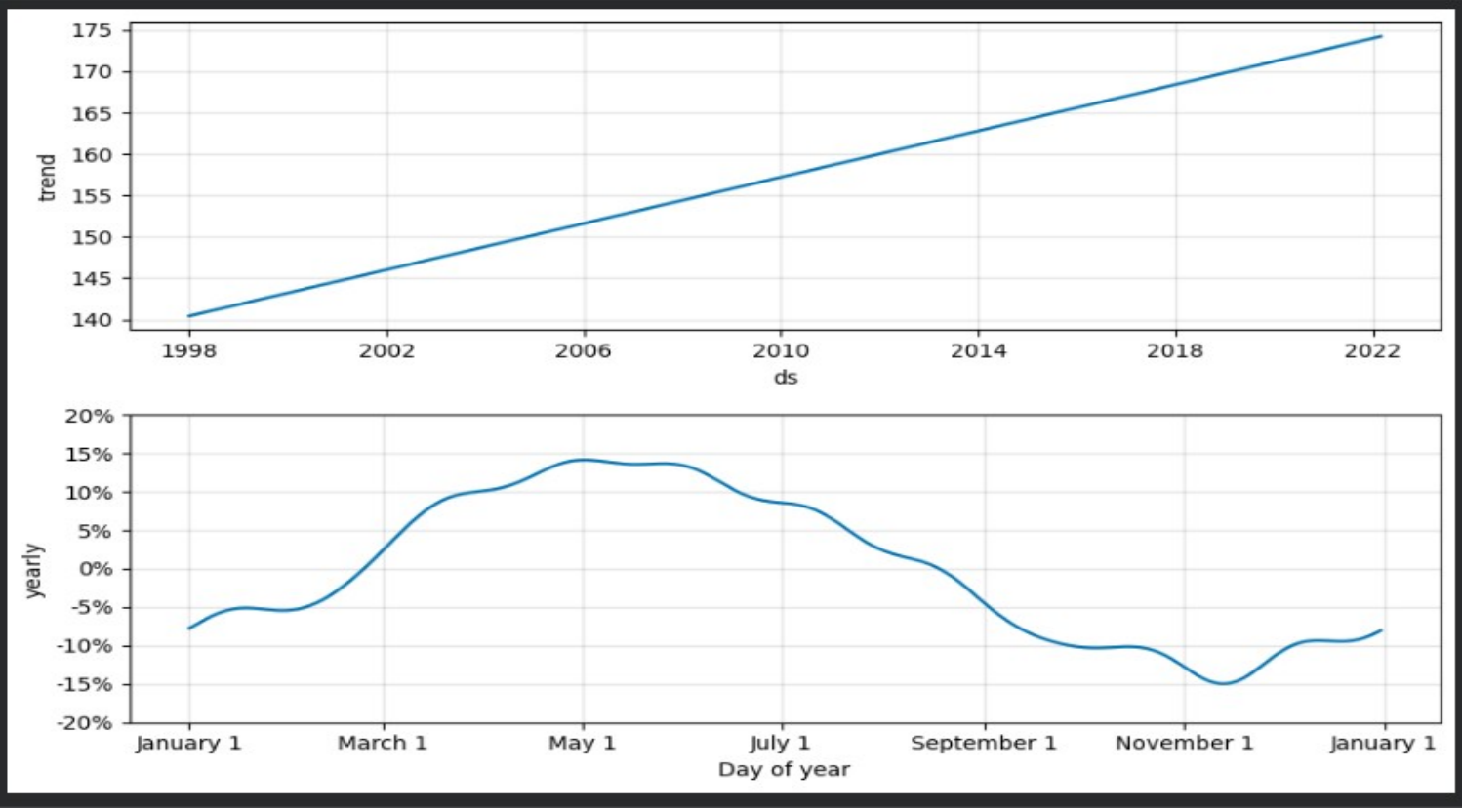
```
futuro = modelo.make_future_dataframe(periods = 12, freq = 'MS') # Especificamos que queremos una predicción de 12 meses, al princ
predicciones = modelo.predict(futuro) # Utilizamos la funcion de predicciones.
modelo.plot(predicciones)
plt.title(f'Prediccion: {objetivo}', fontsize=16) # Montamos la grafica como ya se ha explicado anteiormente.
plt.xlabel('Fecha')
plt.ylabel('htm **3')
plt.show()
```



Esta primera gráfica nos muestra como prophet ha podido ajustar su predicción a los datos recogidos anteriormente.

La linea azul muestra la predicción de prophet mientras que los puntos negros son los datos base, prophet ha hecho un seguimiento correcto de dichos datos. Cuando se acaban los puntos negros es donde empieza la predicción de prophet sobre un plazo de 12 meses con su respectivo margen de error representado por la sombra azul.

```
modelo.plot_components(predicciones) # La prediccion de arriba, es como ha ido el embalse a lo largo de los años.
plt.show() # La prediccion de abajo, indica los meses donde mas cantidad de agua hay y que mes/s hay menos.
```



En la gráfica superior podemos ver que ha seguido una dirección a largo plazo. La linea azul representa la salud de los embalses, podemos observar que es estable y que es linear esto descarta perdidas de agua a largo plazo independientemente de las lluvias anuales.

En la segunda gráfica podemos observar el ciclo anual que ha aprendido. El modelo entrenado ha deducido matemáticamente que el punto donde más agua hay comprende entre finales de marzo principios de abril y mayo los cuales son meses de primavera y hay muchas precipitaciones, y que el punto más bajo comprende entre septiembre y noviembre que es justo cuando acaba el ciclo de estiaje. Esto valida que prophet se basa en la hidrología real.

Hidrología: Es la disciplina científica que estudia las aguas del planeta, los océanos, la atmósfera y la superficie terrestre (Concepto.de, s.f.).

PREGUNTAS

1. ¿Qué embalse tiene más diferencia en su máximo y mínimo de agua?

Tras analizar y organizar el dataframe puedo responder que el embalse con mas agua es el RioLozoya_ElAtazar.

Con una variación de 275,66 hm^3

Sus niveles de agua han ido variando, su mínimo histórico es de 148,22 hm^3 y su máximo histórico es de 423,88 hm^3

2. ¿Cuándo se produce sequía? si esta se llega a producir.

Según la predicción realizada con mi modelo de prophet no habrá sequía en los próximos 12 meses .
Para que llegara a producirse una sequía, habría que bajar de la media calculada sobre los datos históricos, la cual no se cruza. Se producirá una bajada después del verano pero los niveles se mantendrán aceptables sin llegar a entrar en una zona critica.

3.- ¿Podemos asegurar una respuesta segura a la pregunta anterior con los datos que tenemos?

No, no se puede asegurar una respuesta segura.

Prophet trabaja con intervalos en los que confía y solo ofrece probabilidades matemáticas, no algo que va ocurrir.

El dataframe en el que he basado el entrenamiento solo contiene datos pasados, no contiene cambios de temperaturas o el consumo humano.

Aunque es cierto que he limpiado el dataframe original los datos originales contenían errores, los cuales pueden producir desviaciones en las predicciones.

CONCLUSIONES

La implementación de este modelo de prophet entrenado podría llegar a transformar el nivel de operación del Instituto Nacional de Meteorología.
Esto se debe a que actualmente las acciones que se delegan sobre la sequía son de inversiones constantes en cuencas hidrográficas, la cual es una medida de seguridad robusta pero imposible de mantener a largo plazo ya que solo en 2023 se gastaron 17.190 millones de euros en dichas inversiones (la Moncloa, 2023), con una medida de predicciones se podría actuar de manera casi automática esa inversión puntal se reduciría considerablemente incluso podríamos hallar nuevas formas de poder recoger grandes cantidades de agua provenientes de precipitaciones constantes.

Mi intervención humana para poder mejorar el rendimiento y optimizar el modelo entrenado ha sido crucial.
Si hubiésemos entregado los datos originales a la inteligencia artificial “en crudo”, le habría sido imposible hacer una predicción precisa o fiable, ya que la integridad de los datos originales estaban comprometidos por un abuso constante de malas practicas, mi labor de filtrado, limpieza y validación fueron unos pasos obligatorios para que el entrenamiento fuese exitoso.
Durante la fase de entrenamiento no usé los parámetros que vienen por defecto con prophet, ya que a medida que avanzaba me di cuenta de que una variación en los niveles de agua no es constante si no que depende de factores como el cambio climático, por esta razón me decante por usar la opción multiplicativa y no la aditiva, como resultado hubo una mejora drástica en la curva de predicción.

El mayor reto técnico que se puede sufrir en una predicción como esta, es la integridad y seguridad de los datos.
Esto se debe ha que si hubiese alteraciones malintencionadas destrucciones de archivos originales o una simple perdida provocaría un aprendizaje erróneo por parte del modelo. Por ello siempre hay que tener un sistema de seguridad impecable para los archivos originales y hacer predicciones a partir de las copias de los mismos.

Confiar ciegamente en la inteligencia artificial sería un error garrafal ya que por un solo fallo en calculo pondría en riesgo de sequía a mas de 3 millones de habitantes, no solo eso estaríamos violando principios éticos tales como reputacionales, estratégicas, seguridad y responsabilidad (Deloitte, s.f.).

BIBLIOGRAFIA

Concepto.de. (s.f.). *Concepto de Hidrología*. Disponible en: https://concepto.de/hidrologia/ [Consulta: 27 de enero de 2026].

DataCamp. (05-11-2025). *Facebook Prophet Tutorial: Forecasting with Python*. Disponible en: https://www.datacamp.com/es/tutorial/facebook-prophet [Consulta: 27 de enero de 2026].

DataCamp. (21-11-2024). *Loc vs iloc in Pandas*. Disponible en: https://www.datacamp.com/es/tutorial/loc-vs-iloc [Consulta: 27 de enero de 2026].

Deloitte. (26-12-2024). *Riesgos éticos en el uso de la Inteligencia Artificial*. Disponible en: https://www.deloitte.com/latam/es/services/risk-advisory/perspectives/riesgos-eticos-en-el-uso-de-la-inteligencia-artificial.html [Consulta: 27 de enero de 2026].

Exploratory.io. (22-02-2020). *Time Series Forecasting with Prophet: Additive vs Multiplicative*. Disponible en: https://exploratory.io/note/kanaugust/Time-Series-Forecasting-with-Prophet-Additive-vs-Multiplicative-aGB3gVk2gO [Consulta: 27 de enero de 2026].

Factorial. (10-02-2025). *Qué es ISO 8859-1 (Latin-1)*. Disponible en: https://factorial.es/blog/iso-88591/#que_es_iso_8859_1 [Consulta: 27 de enero de 2026].

Interactive Chaos. (s.f.a). *Dataframes: la función concat*. Disponible en: https://interactivechaos.com/es/manual/tutorial-de-pandas/dataframes-la-funcion-concat [Consulta: 27 de enero de 2026].

Interactive Chaos. (s.f.b). *El método dropna*. Disponible en: https://interactivechaos.com/es/manual/tutorial-de-pandas/el-metodo-dropna [Consulta: 27 de enero de 2026].

Interactive Chaos. (s.f.c). *El método head*. Disponible en: https://interactivechaos.com/es/manual/tutorial-de-pandas/el-metodo-head [Consulta: 27 de enero de 2026].

Kanaries. (19-08-2023). *Pandas DataFrame loc*. Disponible en: https://docs.kanaries.net/es/topics/Pandas/dataframe-loc [Consulta: 27 de enero de 2026].

Kanaries. (14-11-2025a). *Pandas Read CSV: Guía completa*. Disponible en: https://docs.kanaries.net/es/topics/Pandas/pandas-read-csv [Consulta: 27 de enero de 2026].

Kanaries. (10-01-2025b). *Pandas Rename Column*. Disponible en: https://docs.kanaries.net/es/topics/Pandas/pandas-rename-column [Consulta: 27 de enero de 2026].

La Moncloa. (12-09-2023). *Resumen del Consejo de Ministros (12.09.2023)*. Gobierno de España. Disponible en: https://www.lamoncloa.gob.es/consejodeministros/resumenes/paginas/2023/120923-rp-cministros.aspx [Consulta: 27 de enero de 2026].

Liora. (s.f.a). *Numpy: la biblioteca Python*. Disponible en: https://liora.io/es/numpy-la-biblioteca-python [Consulta: 27 de enero de 2026].

Liora. (s.f.b). *Pandas Python*. Disponible en: https://liora.io/es/pandas-python [Consulta: 29 de enero de 2026].

Liora. (s.f.c). *Seaborn: la herramienta de data visualization Python*. Disponible en: https://liora.io/es/seaborn-la-herramienta-de-data-visualization-python [Consulta: 27 de enero de 2026].

Liora. (s.f.d). *Todo sobre Matplotlib*. Disponible en: https://liora.io/es/todo-sobre-matplotlib [Consulta: 27 de enero de 2026].

Ministerio para la Transición Ecológica y el Reto Demográfico (MITECO). (s.f.). *Cambio climático e inundaciones*. Disponible en: https://www.miteco.gob.es/es/agua/temas/gestion-de-los-riesgos-de-inundacion/planes-gestion-riesgos-inundacion/cambio-climatico-e-inundaciones.html [Consulta: 27 de enero de 2026].

Pandas Development Team. (s.f.a). *pandas.DataFrame.fillna*. PyData. Disponible en: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html [Consulta: 27 de enero de 2026].

Pandas Development Team. (s.f.b). *pandas.DataFrame.replace*. PyData. Disponible en: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.replace.html [Consulta: 27 de enero de 2026].

Pandas Development Team. (s.f.c). *pandas.pivot_table*. PyData. Disponible en: https://pandas.pydata.org/docs/reference/api/pandas.pivot_table.html [Consulta: 27 de enero de 2026].

Pandas Development Team. (s.f.d). *pandas.to_numeric*. PyData. Disponible en: https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html [Consulta: 27 de enero de 2026].

Planeta Chatbot. (07-05-2025). *Primeros pasos en predicción de series temporales de datos con Facebook Prophet*. Disponible en: https://planetachatbot.com/primeros-pasos-en-prediccion-de-series-temporales-de-datos-con-facebook-prophet/ [Consulta: 27 de enero de 2026].

RDRR.io. (22-01-2026). *fit.prophet: Fit the Prophet model*. Disponible en: https://rdrr.io/cran/prophet/man/fit.prophet.html [Consulta: 27 de enero de 2026].

W3Schools. (s.f.). *Pandas DataFrame ffill()*. Disponible en: https://www.w3schools.com/python/pandas/ref_df_ffill.asp [Consulta: 27 de enero de 2026].